

Week 1 - Day 2a: Variables, Printing, Commenting, Conditionals

You will be working on a series of exercises for the material covered in lecture 2a, including:

- **Printing**
- **Variables**
- **Variable reassignment**
- **Comments**
- **Conditionals**

Exercise 1: Printing []

Printing is one of the most fundamental functions to use while programming. Take a guess at the output of each cell before running it.

1.1 What is the output of `print(True)`?

```
# Check your guess by running this cell  
print(True)
```

1.2 What is the output of the following block of code?

```
print("Hello Friends")  
print(-234.23)  
print(34 + 3)  
  
# Check your guess by running this cell  
print("Hello Friends")  
print(-234.23)  
print(34 + 3)
```

Like values and expressions, you can also print variables. This is often helpful to clear confusion and uncover mistakes by telling you the value of the variable at the time of printing.

1.3 What is the output of this code block?

```
var = 10  
print(var)  
  
# Check your guess by running this cell  
var = 10  
print(var)
```

There are some caveats to printing in Jupyter. Specifically, the last value of a Jupyter cell is printed without an explicit print statement. However, this is only true for the last line. All other lines won't print without explicit print statements.

1.4 What is the value of the following code block?

```
var2 = "Hello"
var2

# Check your guess by running this cell
var2 = "Hello"
var2
```

1.5 What is the value of the following code block?

```
var3 = "World"
var3
32
var4 = "Hello World"

# Check your guess by running this cell
var3 = "World"
var3
32
var4 = "Hello World"
```

Exercise 2: Variable Assignment []

We will start with simple variable assignment. For each of the three cells below you will assign a value (of different types) to a variable.

2.1 Assign the value 523798 to variable x.

```
# TODO: Your code here
```

2.2 Assign the value True to variable x.

```
# TODO: Your code here
```

2.3 Assign the value "this is a string" to variable x.

```
# TODO: Your code here
```

Exercise 3: Variable Reassignment []

As you learned in lecture, you can reassign a value to a variable that was already assigned a value - this is called **variable reassignment**.

Now, for each of the cells below,

1. Guess what the output will be
2. Check your answer by typing (or copying and pasting) the code into the code cell and run the code cell.

3.1 Guess the output and then run the following code block in the code cell.

```
x = 7
x = 3
print(x)

# TODO: Your code here
```

3.2 Guess the output and then run the following code block in the code cell.

```
x = 7
y = x
print(y)

# TODO: Your code here
```

3.3 Guess the output and then run the following code block in the code cell.

```
x = 7
y = x
print(x)

# TODO: Your code here
```

3.4 Guess the output and then run the following code block in the code cell.

```
x = 3
y = x
x = 4
print(y)

# TODO: Your code here
```

3.5 Guess the output and then run the following code block in the code cell.

```
x = 3
y = x
x = 4
print(x)

# TODO: Your code here
```

3.6 Which of the following options are the correct way to swap two elements `m` and `n`'s values.

For example, `m = 3` and `n = 2` initially, and after certain operations, `m = 2` and `n = 3`

```
# A
m = 3
n = 2
m = n
n = m

# B
m = 3
n = 2
m = n
temp = m
n = temp

# C
m = 3
n = 2
temp = n
n = m
m = temp

# D
m = 3
n = 2
temp = m
n = temp
m = n
```

Exercise 4: Variable Assignment with Variables []

Next, we will work with variable assignment using other variables.

As previously, for each of the following code blocks, guess what the output will be before executing the code. See if anything surprises you!

4.1 Guess the output and then run the following code block in the code cell.

```
x = "hello"
y = " world"
x + y

# TODO: Your code here
```

4.2 Guess the output and then run the following code block in the code cell.

```
x = "hello"
y = " world"
z = x + y
x = "bye"
z
```

```
# TODO: Your code here
```

4.3 Guess the output and then run the following code block in the code cell.

```
x = True
y = 3
z = x + y
z
```

```
# TODO: Your code here
```

4.4 Guess what the output of each of the variables `x`, `y` and `z` would be and then run the code to check.

```
x = 12
y = 5
z = y
y = 12
x = 1 + z
z = x + y + z
```

```
x, y, z
```

```
# TODO: Your code here
```

Exercise 5: Manual Variable (Re)-Assignment []

To get some more practical experience with variables, we will ask you to write some code yourself step-by-step.

5.1 Define a variable `x`, that has a value of 5.

```
# TODO: Your code here
```

5.2 Define a variable `y`, that has a value of `x`.

```
# TODO: Your code here
```

5.3 What is the value of `y`?

5.4 Change `x` to 4.

```
# TODO: Your code here
```

5.5 What is the value of `y`?

5.6 Define a variable `z`, that has a value of the sum of `x` and `y`

```
# TODO: Your code here
```

5.7 Assign `x` another value, `12`. What are the values of `y` and `z`?

```
# TODO: Your code here
```

Exercise 6: Unassigned Variables []

What do you think happens when you try to do a computation with a variable which you have not yet given a value? Let's find out!

6.1 Read the below code cell and guess the output of the code cell. Do you expect an error or not? If yes, what kind of error do you expect? Check your guess by running the code cell.

```
a = 5
x = a + b
x
```

```
# TODO: Your code here
```

6.2 What do you need to add to the code so that it does not give an error? Implement the change in the below cell.

```
# TODO: Your code here
```

Exercise 7: (Tricky) Variable Names []

As you have learned in lecture, you have quite a lot of freedom in naming your variables in Python. However, there are some constraints defined by the Python syntax. The rules for variable names in Python are:

- A variable name **must start with a letter or the underscore** character
- A variable name can **only contain alpha-numeric characters and underscores** (A-z, 0-9, and `_`)
- Variable names are case-sensitive (`age`, `Age` and `AGE` are three different variables)
- A variable name **cannot be any of the Python keywords**

Thus, a variable can have a short name (like `x` and `y`) or more descriptive names, such as `age`, `name` or `total_volume`. Typically, for larger projects and codebases descriptive variable names help with code readability and maintainability.

7.1 Given the below code, what is the value of the variable `seven`?

```
seven = 8
```

```
# TODO: Your code here
```

7.2 Given the below code, are the variables `shiro` and `Shiro` equivalent? Reason for your answer.

```
shiro = 10
Shiro = 12

# TODO: Your code here
```

Exercise 8: Commenting Code []

Adding comments to complicated code is generally good practice, because it helps other people (and also yourself if you read your code in the future) to understand your code more easily. Understandable variable names are also a critical part of good code.

Remember, you do **not** need to comment for every line, but only lines that may be confusing to read.

The following code (correctly) computes the area of the shape which consists of a square, triangle and semicircle.

```
a = 3
s = a**2
d = 0.5*3.14*(a/2)**2
t = 3**0.5 / 4 * a**2
aa = s + d + t
aa
```

However, it is not very easy to understand what happens in each line.

8.1 Copy the code above and add comments to explain what each line of code does.

```
# TODO: Your code here
```

Nice! This is already easier to understand. One other thing that helps make code easier to understand is to use descriptive variable names, i.e. names that represent what is stored in the variable. For example, instead of `aa`, you could use `total_area`.

8.2 Modify the code above by using descriptive variables names, i.e. rename `a`, `s`, `d`, `t`, and `aa`.

```
# TODO: Your code here
```

Exercise 9: Conditionals []

Conditionals are a fundamental concept in programming. They allow you to execute different code depending on whether a condition is true or false. In Python they are implemented using `if`, `elif` and `else` keywords.

9.1 What will be the output of the following code snippet?

```
x = 10
if x > 5:
    print("Greater than 5")
elif x > 8:
    print("Greater than 8")
else:
    print("Else")
```

Problems 9.2 to 9.6 rely on the following information:

Different **ages** correspond to different **school** in Ethiopia. This shows these categories:

Age Range	Education Level
0 - 5	No School
6 - 13	Elementary School
14 - 18	High School
19 - 22	University
23 - 100	No School

How can we figure out which school someone should go to based on their age?

9.2 First, define a variable `daniel_age` that stores his age (any integer between 0 and 100). Then, write a conditional to check if Daniel is in `Elementary School`. If he is, print `Elementary School`. Otherwise, do nothing.

```
# TODO: Your code here
```

9.4 Similar to above, how would you write a **conditional** to check if Daniel is in the `"No School"` category? Make sure to verify both cases (`"Elementary School"` and `"No School"`) and if the age is in those ranges, **print** the string `"No School"` and `"Elementary School"` respectively.

```
# TODO: Your code here
```

9.5 Write a conditional that prints `"No School"` if the person is not in school or `"School"` if the person is in school.

```
# TODO: Your code here
```

9.6 Finally, using the code pieces you just wrote, write a **conditional** that prints the category of school they are in (string) for any age. Change the age to test your code.

```
# TODO: Your code here
```

9.7 The sign of a number is +1 if the number is positive, -1 if the number is negative, and 0 if it is 0. Define a variable `x`, and write code that computes and prints its sign.

Hint: Use conditionals

```
# TODO: Your code here
```

9.8 Define another variable `y`, and write code that computes and prints the absolute value of that number. Use conditionals to complete this task!

The absolute value of a number is the number itself, if it is positive, and its negation if it is negative (i.e. the number turned positive). Ex. the absolute value of 2 is 2 and the absolute value of -2 is 2 as well.

```
# TODO: Your code here
```

9.9 Using the code from above, write code that takes two variables `x` and `y` and computes and prints the number `y` but with the sign of `x`.

For example:

- `x = 16` and `y = -2` --> 2
- `x = 16` and `y = -14` --> 14
- `x = -16` and `y = -2` --> -2
- `x = -16` and `y = 2` --> -2
- `x = 0` and `y = 2` --> 0

```
# TODO: Your code here
```

Exercise 10: Assignment with Mutation []

In Python, you can also mutate a variable while assigning it. Here are some of the operators you can use for the following exercises:

- `+=`
- `-=`
- `*=`
- `/=`
- `%=`

How do these operators work? For example, `x += 3` adds 3 to the current value of `x` and assigns the new value to the variable `x`. This works similarly for the other operators.

10.1 Given the below code, what are the values of `x` and `y`?

```
x = 99
x /= 10
x *= 50
y = x
```

```
y += 5
x -= y
y %= x
```

Try to do it in your head (or on paper) first and write your guesses in this Markdown cell. Then run the code cell to check your answers.

```
# TODO: Your code here
```

10.2 Define a variable `x` that has the value `10` times `20`.

```
# TODO: Your code here
```

Now increment `x` by 3.

```
# TODO: Your code here
```

Can you increment `x` by 3 in a different way (using a different operator)?

```
# TODO: Your code here
```

10.3 Find the remainder of `78` divided by `14` using one operator from lecture. For example, the remainder of 16 divided by 5 is 1 because $16 = 5 * 3 + 1$.

```
# TODO: Your code here
```

10.4 Given `x`, make `x` 10 times smaller using one operation. Return a value that has the type `int`.

```
# TODO: Your code here
```

10.5 Modify the variable containing `"Shiro"` so that it contains the new value `"ShiroShiroShiro"` using a single operator

```
# TODO: Your code here
shiro = "Shiro"
```

10.6 Modify the variable containing `"Shiro"` so that it contains the new value `"Shiro3"` using a single operator

```
# TODO: Your code here
shiro = "Shiro"
```

If you've completed all of the exercises so far, congratulations! There are supplementary exercises you can do now. Ask a TA to show you how to get access to them.